

Lab 05: Version Control Systems (Git & GitHub)

Part A: Installation & Prerequisites

Complete these steps before starting the lab.

1. Install Git

- **Download:** Go to git-scm.com/downloads and download the version for Windows.
- **Install:** Run the installer. **Crucial:** When asked about adjusting your PATH environment, ensure "Git from the command line and also from 3rd-party software" is selected. For all other options, keep the **defaults**.

2. Install VS Code Extension

- **Open VS Code.**
- **Go to Extensions:** Click the square icon on the left sidebar (**Ctrl+Shift+X**).
- **Search:** Type "**GitHub**".
- **Install:** Select all the official extensions named Github and click **Install**.
- *Why?* This integrates GitHub directly into your editor, allowing you to push/pull code without leaving VS Code.

Part B: Concept Walkthrough (Instructor Demo)

This section outlines the workflow demonstrated during the lecture.

1. Configuration (First-Time Setup)

Before tracking any code, you must identify yourself. This information is embedded in every commit.

Step: Open the VS Code Terminal (**Ctrl** + **`**) and run the following commands:

Bash

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

2. Initialization (Starting a Repo)

- **Step:** Create a folder and open it in VS Code.
- **Action:** Go to the **Source Control** tab (Branch icon on the left). Click **Initialize Repository**.
- **Concept:** This is the equivalent of running **git init** in the command line.

3. Staging and Committing (The Save Loop)

Git has a two-step save process: Staging (preparing files) and Committing (saving the snapshot).

- **Stage:** Hover over a changed file in the list and click the **+** (**Plus**) icon. This is equivalent to **git add**.
- **Commit:** Type a descriptive message in the box and click **Commit** (Checkmark). This is equivalent to **git commit -m "message"**.

4. Branching (Parallel Development)

- **Step:** Click the branch name (usually **master** or **main**) in the bottom-left corner of the VS Code status bar.
- **Action:** Select **+ Create new branch** and name it.
- **Concept:** This isolates your new code from the main project until it is ready.

5. Publishing (Pushing to GitHub)

- **Step:** Click the **Publish Branch** (cloud icon) button in the Source Control tab.
- **Action:** Select "Publish to GitHub public repository".
- **Concept:** This automates creating a remote repository and pushing your local code.

Part C: Git Command Cheat Sheet

While VS Code provides buttons, the following commands are essential for troubleshooting and manual control.

VS Code vs. Command Line Table

Action	VS Code GUI Action	CLI Command
Start	Click "Initialize Repository"	<code>git init</code>
Track File	Click (+) next to file name	<code>git add <file></code>
Save	Click Commit (Checkmark)	<code>git commit -m "msg"</code>
Upload	Click "Sync Changes" / "Publish"	<code>git push</code>

Essential CLI Commands

Setup & Status

- `git status`: Shows modified, staged, and untracked files.
- `git log --oneline`: Shows a simplified history of commits.

Branching & Merging

- `git branch`: Lists all local branches.
- `git checkout -b <name>`: Creates and switches to a new branch.
- `git checkout master`: Switches back to the master branch.
- `git merge <branch>`: Merges a specific branch into your current branch.

Remote (GitHub)

- `git clone <url>`: Downloads a remote repository to your local machine.
- `git pull`: Updates your local code with changes from the remote repository.

Part D: Student Lab Task

Scenario: You are a Junior Developer at a software firm. Your task is to safely develop a "Contact Us" feature for the company website in a separate branch, then merge it into the main project.

Step 1: Setup & Initialization

1. Create a folder on your desktop named `Lab05_StudentId`.
2. Open this folder in VS Code.
3. Initialize Git (using the button or `git init`).
4. Create a file named `home.html`.
5. Add the code: `<h1>Welcome to My Website</h1>`.
6. **Stage and Commit** this file with the message: "Initial commit: Added home page".

Step 2: Feature Branch Development

Never code new features directly on the master branch.

1. Create a new branch named `feature-contact`.
2. Switch to this branch (ensure the bottom-left status bar says `feature-contact`).
3. Modify `home.html` by adding: `<p>Contact us at: email@test.com</p>`.
4. **Stage and Commit** this change with the message: "Added contact info".
5. *Check:* Switch back to the `master` branch. The contact info should disappear. Switch back to `feature-contact`.

Step 3: Merging Changes

Your feature is ready to go live.

1. Switch to the `master` branch.

2. Merge the feature branch into master.
 - *Hint:* You can use `git merge feature-contact` in the terminal OR use `Ctrl+Shift+P` > "Git: Merge Branch" in VS Code.
3. Verify that `home.html` now contains the contact info while you are on the master branch.

Step 4: Push to GitHub

1. Log in to GitHub and create a new **Public** repository named `Lab05-Git-Practice`.
2. Connect your local folder to this remote repository.
3. Push your code to GitHub.

Bonus: The .gitignore File

1. Create a file named `secrets.txt` in your folder.
2. Create a file named `.gitignore`.
3. Add `*.txt` inside the `.gitignore` file.
4. Verify that `secrets.txt` is ignored by Git and cannot be staged.